使用说明

一、 交付文件清单

二、 核心 API 接口说明

1. AES-CTR 加解密（对称加密）

2. CMAC 认证（防篡改）

三、 安全红线

四、 测试与验收标准（交接前请让对方跑一遍）

一、 交付文件清单

文件名称	类型	说明
`types.h`	必需	基础类型定义（`Bytes`，`Key128`，`Block128`，`MacTag`）
`aes.h`	必需	AES-128 底层接口及 C++ 类接口声明
`aes.cpp`	必需	AES-128 算法实现及 C++ 适配层
`cmac.h`	必需	CMAC 认证接口声明
`cmac.cpp`	必需	CMAC (RFC 4493) 算法实现
`test_cmac.cpp`	参考	RFC 4493 标准测试向量（仅供验证用）
`cmac_benchmark.cpp`	参考	性能测试（仅供验证用）

⚠️ **编译环境要求**：支持 C++11 及以上即可。**请务必在 Release 模式下编译**（Debug 模式速度仅为 1 MB/s，Release 可达 90+ MB/s）。

二、 核心 API 接口说明

所有数据类型均使用 `uint8_t` / `Bytes`（`std::vector<uint8_t>`），**严禁使用 `char*`**（`char` 有符号位问题，会导致中文/高位字节解密乱码）。

1. AES-CTR 加解密（对称加密）

cpp

Plain Text

```
#include "aes.h"

AES aes;
Key128 sessionKey = { /* 16字节会话密钥 */ };
Block128 nonce = { /* 16字节随机数，每条消息必须不同 */ };

// 原地加解密（加密和解密调用的是同一个函数）
aes.ctrXor(sessionKey, nonce, dataPtr, dataLen);
```

2. CMAC 认证（防篡改）

cpp

Plain Text

```
#include "cmac.h"

CMAC cmac;
Key128 macKey = { /* 16字节认证密钥 */ };
Bytes message = { /* 待认证数据 */ };

// 生成标签（16字节）
MacTag tag = cmac.generate(macKey, message);

// 验证标签（返回 true/false）
bool isValid = cmac.verify(macKey, message, receivedTag);
```

三、 安全红线

1. **Nonce 绝对不可重复**：在同一个 `sessionKey` 下，绝不允许使用相同的 `Nonce`。CTR 模式一旦 Nonce 重复，密钥流就会重复，攻击者可以直接异或两条密文得到明文异或关系。建议双方维护一个递增的 128 位计数器，或使用密码学安全随机数生成器（CSPRNG）。
2. **Encrypt-then-MAC**：必须先加密，再对密文和 Nonce 算 CMAC；接收方必须先验 CMAC，再解密。绝对不要反过来。
3. **密钥管理**：`sessionKey` 和 `macKey` 建议使用不同的密钥（密钥分离），不要用同一个密钥既做加密又做认证。密钥需要提前通过安全信道协商好。
4. **Release 模式**：调试阶段可以用 Debug，但正式发布或压测请务必切 Release，否则性能会有百倍差距。

四、 测试与验收标准（交接前请让对方跑一遍）

请将 `test_cmac.cpp` 加入项目编译，并运行。如果终端输出以下内容，说明密码学模块 100% 正确无误：

text

Plain Text

```
[PASS] empty
[PASS] 16B
[PASS] 40B
[PASS] 64B
4/4 passed
```

测试通过后，即可将 `test_cmac.cpp` 和 `cmac_benchmark.cpp` 从生成中排除，开始正式的 TCP 网络开发。